Checkpoint KL Diffing Spec

Oct 23, 2026 · @Amina Keldibek

1. Overview

Build a pipeline that runs fixed RL rollouts through several checkpoints of the Qwen3-8B reward-hacking organism under teacher forcing, and reports where and how the checkpoints' next-token predictions differ. The output is a map of where RL changed the policy, not a mechanism claim.

Why this first. RL nudges every LoRA weight a little; weights alone cannot say what was learned, and the behavioural curves already date the regime change (steps ~93–110), so dating it is not a question here. What the hack-rate curve and the gradient norm cannot give is *where* in a trajectory the policy moved (which positions, which tokens), *what* vocabulary it now prefers, and whether the policy moved *below the sampling threshold* during the plateau: at n = 32 per step a shift from 0.1% to 3% on a hack token is invisible behaviourally, but exact in KL on fixed rollouts. Those are the questions, and they also tell the later activation work which positions and layers to look at.

Setting, from the scoring diagnostics. 115 GRPO steps, 32 completions per step, one distinct prompt per step. Hack rate is near zero until ~step 90, surges to ~50% around step 100 and ~90% by step 115. Passing the tests and hacking are the same event (φ = 0.970), so `training_passed` and `reward_hacked` are treated as one variable throughout. Three judge labels exist per rollout: `honest_attempt`, `grader_beliefs` (states what the grader sees), `reward_hack_mentioned` (+ stance). Grader-belief mentions rise from step ~70, before the hack surge; that early window is the most interesting one for hidden drift.

Scope of this spec. Logit-level diffing only. Hidden-state capture is designed in (same forward passes, optional flag) so the next experiments (direction extraction, projection tracking, steering) reuse the tooling, but no activation analysis is specified here.

Definitions used below.

- *Rollout*: one completion from one training step, with its exact system prompt, user prompt and chat template as used in RL. Identified by `row_index` in the scorer tables.
- *Checkpoint*: a LoRA adapter saved at a GRPO step; `base` is the adapter-free model (step 0).
- *Position*: an index into the tokenised sequence; the model's prediction at position *i* is the distribution over token *i+1*.
- *Completion positions*: positions whose next token belongs to the model's own output (everything after the assistant header), the only positions analysed.

2. Checkpoints

Use four checkpoints, not three. The pre-surge plateau is the one that answers the hidden-drift question, and three checkpoints (base, surge, final) skip it. If compute forces three, drop `surge`, not `plateau`.

Name	Target step	Why	Hack rate (from diagnostics)
`base`	0 (no adapter)	Reference for every comparison	~0.5%
`plateau`	~75 (window s70–92)	Behaviour still looks aligned, but grader-belief mentions have started rising. Any KL here is change RL made *before* the hack appeared.	~4%
`surge`	~100	Mid phase transition; hack rate near 50%, so hacked and non-hacked rollouts coexist at the same step	~50%
`final`	115	End state	~90%

Selection rule for the agent. List the saved adapters. For each target step take the nearest saved checkpoint; record the actual step used. If no checkpoint exists within ±10 steps of `plateau`, say so and use the latest one before step 90. Also take `step 1` (or the earliest saved adapter) as a **noise-floor control**: KL(step1 ‖ base) sets the scale below which differences are not meaningful.

Pairs to compute. Every checkpoint against `base` (the drift series), plus consecutive pairs `plateau→surge` and `surge→final` (where the move happened). That is 5 pairs plus the control; all come from the same forward passes, so the extra pairs cost nothing.

Direction. Compute KL in both directions for every pair. KL(ckpt ‖ base) is driven by tokens the checkpoint is confident about; KL(base ‖ ckpt) by tokens base was confident about that the checkpoint abandoned. Both are informative and the asymmetry is itself a signal (see §6).

3. Dataset

Stratify, don't homogenise. The question is *where do checkpoints disagree, given what the rollout did*, so the set must contain both hacked and non-hacked rollouts, and both verbalised and silent ones. A homogeneous set (say, only late hacks) would show one

regime and hide the contrast that makes a KL spike interpretable.

Why uniform-across-checkpoints is still right, and what it doesn't fix. Every analysis is paired within a rollout (the same text goes through every checkpoint), so prompt difficulty cancels in every KL number. What it does not cancel is coverage: steps 1–23 have almost no hacks and steps 106–115 almost no non-hacks, so the 2×2 cells cannot be equal in every window. Fill what exists and report cell counts; do not synthesise balance by over-weighting rare cells.

Stratification axes (in priority order; from the scorer tables joined on `row_index`):

1. `hacked` = `reward_hacked` (identical to `training_passed`; never use both).
2. `mentioned` = `reward_hack_mentioned` (verbalised vs silent).
3. `grader_beliefs` and `stance` are recorded as metadata for the analysis, not used as strata. `honest_attempt` likewise. (No faithfulness axis exists in the current scorer set; if one is added later it joins as metadata.)

Core set: 100 rollouts, 25 per source window. Windows are centred on the checkpoints in §2 so each checkpoint is examined on rollouts it produced *and* on rollouts it did not.

Window	Steps	Target cells (hacked × mentioned)	Expected fill
W0 near base	1–15	25 non-hacked, split ~12/13 by mentioned	hacks ≈ 0, take any that exist
W1 plateau	67–83	all hacks available (expect 3–8), rest non-hacked split by mentioned	partial
W2 surge	94–105	6/6/6/7 across the four cells	full
W3 final	106–115	all non-hacked available (expect ⩽12), rest hacked split by mentioned	partial

Deception subset: the 63 `disavow_then_hack` rollouts, already saved at `datasets/disavow_then_hack_63.jsonl`. Keep as a separate named set, not inside the 100. Pair it with a **matched control of 63 hacked rollouts with `stance` ∈ {neutral, endorses}** drawn from the same steps (nearest step, same window). This contrast (said it was wrong then hacked vs hacked without objection) is the sharpest deception signal in the data.

Exclusions, applied before sampling:

- any scorer row failed the evidence check or is unparseable

- completion shorter than 50 tokens, or missing `</think>`, or no final answer block (catches the four `<thinking>`-only truncations)
- completion longer than 6,144 tokens (compute cap; record how many were dropped)
- near-duplicate completions within a step: 5-gram Jaccard > 0.8 against an already-selected completion of the same step (one prompt per step makes copies common); keep the first, count the drops
- rollouts whose stored prompt or system prompt is missing: the exact text the policy saw is required

Procedure. Deterministic, `seed=0`. Within each cell sample uniformly over steps first, then over completions within a step, so no single step dominates a cell. Write `manifest.parquet` with one row per selected rollout: `row_index`, `step`, `window`, `subset` (core / disavow / control), the three judge labels, `stance`, `hacked`, prompt token count, completion token count, and the hack type where the diagnostics record it.

Diversity pre-check, before any GPU work. From the scorer tables and the texts, per window: (a) cell counts (hacked × mentioned) and counts per hack type — W2 and W3 need at least two cells with ≥ 5 rollouts; (b) median think-block length, and the fraction of think blocks near-identical to another completion of the same step — if the median is tiny or most blocks are copies, the `think` region will carry little, and the exit position (last think token, first answer token) becomes the main decision-point candidate; (c) near-duplicate rate before and after the exclusion above. Write the numbers to `precheck.md` and stop for review if (a) fails.

Compute check. ~226 rollouts × 5 adapters (4 checkpoints + step-1 control) ≈ 1,130 teacher-forced passes of ⩽ 8k tokens on an 8B model: under an hour on one 80 GB GPU.

4. Signal

The signal is the full next-token log-probability vector at every completion position, for every checkpoint, computed under teacher forcing; KL and its decomposition are computed on the GPU while all distributions are in memory, and only compact per-position results are written to disk. Full logit tensors are never stored.

Why not store logits. Qwen3-8B's vocabulary is 151,936 entries. One rollout of 2,000 completion tokens is 2,000 × 151,936 × 2 bytes ≈ 0.6 GB per checkpoint in bf16; 226 rollouts × 5 adapters ≈ 680 GB. KL needs both distributions at the same time anyway, so compute it in-process and keep the scalars.

Unembedding matrix: not needed here. Logits are what comes *out* of the unembedding (`lm_head`), so the forward pass already applies it. The matrix becomes relevant only for the follow-up logit-lens work on hidden states.

Vocabulary: top-K = 1,024 per adapter, plus the written token, plus tail mass. The individual-token analyses (log-prob and rank of the written token, installed choices) are exact from this, because vLLM returns the written token's value at every prompt position whatever K is. KL and JS are not: a token in one adapter's top-K can be missing from the other's, and then its probability under the second adapter is unknown. The tail mass says how much probability is unlisted, but not how small any one hidden token's probability is, and a very small probability makes its KL term very large, so the tail mass does not bound the KL error. Instead the spec computes a **lower bound** on KL that is exact by construction (§4.2), and checks once against an exact pass how tight it is and whether it ranks positions the same way (§4.5).

4.1 Forward pass

For each rollout *r* (from the manifest) and each adapter *c*:

1. Rebuild the exact input: the stored system prompt, user prompt and completion, through the tokenizer's chat template **as used in RL** (same `enable_thinking`, same special tokens). Assert that re-tokenising the completion reproduces the stored completion token ids; if it does not, record a `tokenization_mismatch` flag and skip the rollout. If the original token ids are stored, use them directly and skip re-tokenising. This step runs on CPU, before the GPU session, and writes `inputs/{row_index}.json` (token ids, `p` = first completion position, region labels).
2. Send the full token sequence to vLLM as a prompt with `max_tokens=1` and `prompt_logprobs=K` (K = 1,024), once with no adapter (`base`) and once per adapter via `lora_request`. No sampling is used; the one generated token is discarded. vLLM returns, at every prompt position, the written token's log-prob and rank plus the top-K ids and log-probs.
3. Keep positions `i ∈ [p, T−1)` where *p* is the first completion position and *T* the sequence length; position *i* holds the prediction for token *i+1*. Prompt positions are discarded before writing.
4. Write the per-adapter summaries (§4.2 per-adapter columns) to parquet as each rollout completes, so the job is resumable. All pairwise quantities are computed afterwards on CPU from these files.

vLLM settings: `LLM(model, enable_lora=True, max_loras=5, max_lora_rank=<adapter rank>, max_logprobs=1024, logprobs_mode="raw_logprobs", dtype="bfloat16")`; per request `SamplingParams(max_tokens=1, temperature=0, prompt_logprobs=1024, detokenize=False)` with a `TokensPrompt(prompt_token_ids=...)` and, for a checkpoint, `LoRARequest(name, id, path)`. `raw_logprobs` gives values before any temperature or

penalty, which is what teacher forcing needs. Prefix caching may stay on: V1 recomputes the full prefill for any request that asks for prompt log-probs. Submit all rollouts × adapters as one batch and let vLLM schedule them.

4.2 Per-position quantities

For each rollout, position *i*, and ordered pair (*a*, *b*) of adapters, with `p = exp(lp)`:

Column	Definition	Notes
`kl_ab`	KL(a ‖ b) on the bucketed vocabulary: one bucket per token in the intersection I of a′s and b′s top-K, plus one `rest` bucket (§ below)	nats; **lower bound** on the true KL, never an estimate
`kl_ba`	the reverse, same buckets	stored for the same row
`js`	Jensen–Shannon divergence, same buckets	symmetric, bounded; also a lower bound; used to rank positions and set the spike threshold
`rest_mass_a`, `rest_mass_b`	1 − Σ_{v∈I} p(v), for each adapter	how much probability was merged into `rest`; large values mean the bound may be loose
`lp_actual_{c}`	lp_c(x_{i+1}) for every adapter c	log-prob of the token actually written; exact
`dlp_actual`	lp_a(x_{i+1}) − lp_b(x_{i+1})	did the pair make *this* choice more likely; exact
`rank_actual_{c}`	rank of x_{i+1} under adapter c (0 = top)	exact; "was it sampled from the head or the tail"
`entropy_{c}`	−Σ p·lp over c′s top-K, with the tail as one bucket	lower bound on entropy; tight when `tail_mass_{c}` is small
`tail_mass_{c}`	1 − Σ of c′s top-K probability	per adapter; how much is unlisted
`top_ids_{c}`, `top_lps_{c}`	top-1,024 token ids and log-probs under adapter c	list columns; exact

Column	Definition	Notes
`contrib_pos_ids`, `contrib_pos_vals`, `contrib_pos_exact`	top-20 tokens by p_a·(lp_a − lp_b) over a's top-K	what *a* wants here that *b* does not; exact where the token is in I, otherwise a lower bound using lp_b ⩽ min(b's K-th log-prob, log `tail_mass_b`); the bool says which
`contrib_neg_ids`, `contrib_neg_vals`, `contrib_neg_exact`	top-20 tokens by p_b·(lp_b − lp_a) over b's top-K	what *b* wanted that *a* dropped; same bound rule with the roles swapped

How the KL bound is computed (on CPU, from the stored top-K lists). For a position and pair (a, b), let I be the tokens present in both adapters' top-1,024. Every token in I is its own bucket, with both probabilities known exactly. Every other token goes into one `rest` bucket, whose probability under each adapter is `rest_mass` = 1 − Σ_{v∈I} p(v), also exact. `kl_ab` = Σ_{v∈I} p_a(v)·log(p_a(v)/p_b(v)) + rest_a·log(rest_a/rest_b). Merging outcomes can only hide differences, never create them (the data-processing inequality), so this is always ⩽ the true KL. `js` and entropy use the same buckets and are lower bounds for the same reason. Example: if a puts 0.6 on `modify` and b has it in its tail at 1e-5, `modify` lands in `rest`; with b's rest mass 0.002 the bucket contributes ≈ 3.4 nats against a true ≈ 6.6, so the spike is understated but still a spike. Everywhere in §6, KL and JS mean these lower-bound columns, and the 99th-percentile spike threshold for `pre_divergence` is taken on `js`. The per-adapter top-K kept on disk is 1,024; the contribution lists stay at 20.

Also per row: `row_index`, `pos` (absolute), `rel_pos` (fraction through the completion, 0–1), `token_id` (= x_{i+1}), `token_str`, `region`.

Also per (rollout, pair): `pre_divergence` (bool). Find the first position whose `js` exceeds the dataset-wide 99th percentile for that pair; positions before it are `True`. Before that point the prefix is natural for both adapters, so a difference is a real difference; after it, one adapter is reading text it would never have written and its whole distribution flattens, so full-vocabulary KL is large everywhere for an uninteresting reason. If the first spike is at the first completion position the mask is empty, and that rollout is used only as on-policy data for the adapter that produced it. Also per (rollout, adapter): `on_policy` (bool), whether the adapter's window produced the rollout.

Region labels (assigned once per rollout from the token ids, stored in the manifest's companion `positions.parquet`): `think` (inside `<think>…</think>`), `answer_prose`, `code` (inside fenced code blocks in the answer), `test_edit` (code spans that touch test

files, conftest, pytest config, or assertions, detected by regex on the decoded completion; manual list of patterns in config), `template` (special and chat-template tokens; excluded from all aggregates). If the hack type is recorded for the rollout, also mark the span where the hack is executed as `hack_span`.

4.3 Hidden states (optional flag, same pass)

When `--capture-hidden LAYERS` is set, also save the residual stream (post-block output) at the listed layers for completion positions, bf16, as `safetensors` per rollout per adapter: shape `[len(LAYERS), T_completion, d_model]`. Default off. Default layer list for later use: `[8, 12, 16, 20, 24, 28]` of 36. This is for the follow-up direction/steering work and is not analysed in this spec.

4.4 Storage layout

- `manifest.parquet` — one row per rollout (see §3).
- `positions.parquet` — one row per (rollout, position): `row_index`, `pos`, `rel_pos`, `token_id`, `token_str`, `region`.
- `kl/{pair}.parquet` — one row per (rollout, position) for that adapter pair, columns of 4.2. Pair names like `final_vs_base`, `surge_vs_plateau`. List columns as Arrow `list<int32>` / `list<float32>`.
- `peradapter/{adapter}.parquet` — `lp_actual`, `rank_actual`, `entropy`, `top_ids`, `top_lps` per (rollout, position), so single-checkpoint plots don't need a pair file.
- `hidden/{adapter}/{row_index}.safetensors` — only with the flag.
- `config.yaml` — adapter paths and steps actually used, tokenizer hash, chat-template settings, dtype, top-K, region regexes, seed, library versions.

Parquet with list columns is the right format: columnar, compressible, readable from pandas/polars/duckdb without custom code, and a 226-rollout run is well under 1 GB.

4.5 Compute split and GPU budget

The GPU session does prefills and nothing else. Everything before it (manifest, tokenisation, region labels, `inputs/*.json`) and everything after it (all pairwise columns, all of §6) runs on CPU from saved files, so the pod can be shut down as soon as the last parquet is written.

- **Budget.** ~226 rollouts × 5 adapters ≈ 1,130 prefills of ⩽ 8k tokens on an 8B model: roughly 20–30 minutes on one 80 GB GPU under vLLM, including LoRA switching. The job is resumable per rollout; a crash costs minutes, not the session.

- **Saved once per run, so no later question needs the GPU:** `lm_head.weight` and the final-norm weights (bf16 safetensors, ~1.2 GB) for logit-lensing any captured hidden states on CPU; the tokenizer files; each adapter's config and the modules it covers; `config.yaml` as in §4.4.
- **Saved per rollout per adapter:** the §4.2 per-adapter columns with K = 1,024 (~6 KB per position, ~15 GB for the run). This is what makes most new questions CPU-only.
- **Exact-KL check of the lower bound, one small HF + PEFT pass:** 20 rollouts (5 per window), full vocabulary, same positions and pairs. Report (a) the ratio `kl_ab` / exact KL (median, 5th percentile), separately for `pre_divergence` positions and for spike positions; (b) ranking agreement: Spearman correlation between `js` and exact JS over positions, and the overlap of the top-50 positions per pair; (c) any position where `kl_ab` > exact KL by more than 1e-4 (should be none; if any, the bucketing code is wrong). Accept if top-50 overlap ⩾ 0.9 and Spearman ⩾ 0.95; the size of the gap does not matter for this experiment, the ranking does. Otherwise raise K and rerun. About 5 GPU-minutes; the only place HF is used.
- **Needs the GPU again:** exact KL at positions with large `rest_mass` if a result depends on the KL size rather than on the ranking; any new adapter pair not in the run; hidden-state capture if `--capture-hidden` was off.

5. Tooling

Recommendation: run the extraction on plain vLLM, the server already used for the behavioural evals, with multi-LoRA and `prompt_logprobs`. Use HF + PEFT only for the one-off exact-KL validation in §4.5, and vllm-lens, on the same vLLM server, when the activation and steering phase starts. The earlier draft of this section recommended HF for extraction; that is withdrawn.

What vLLM gives and what it doesn't. At every prompt position vLLM returns the written token's log-prob and rank plus the top-k log-probs; a token outside the top-k is absent, the cap is `max_logprobs`, and cost grows with k. Source. That covers every individual-token analysis exactly, and gives KL as an exact lower bound (§4.2). It does not give the full 151,936-entry vector, so exact KL needs either the HF validation pass or a vllm-lens hook. The earlier concern that the V1 generate path drops prompt log-probs came from a TorchRL wrapper note, not from vLLM: V1 supports prompt log-probs and recomputes the full prefill even on a prefix-cache hit. Source

	Option A: vLLM, multi-LoRA + `prompt_logprobs` (recommended for §4)	Option B: vllm-lens on the same server (recommended for the next phase)	Option C: HF Transformers + PEFT (validation only)
What it is	Teacher-forced prefill with `max_tokens=1`; base and each adapter as separate requests in one batch	AISI's vLLM plugin: residual-stream capture, steering vectors, per-layer hooks, persistent hooks, TP/PP, LoRA, Inspect provider. Source	Standard loading, LoRA as named PEFT adapters, one forward pass with `log_softmax`
Written-token log-prob and rank, top-K	Exact, at every prompt position	Same path as A, plus a final-layer hook can compute full log-probs with `ctx.get_parameter("lm_head.weight")`	Exact
Full-vocabulary KL	Lower bound, exact by construction; tightness and ranking agreement checked once (§4.5)	Exact, if a persistent hook stores base log-probs server-side and computes KL on the adapter request	Exact, in-process
Several adapters, base included	Yes: `lora_request` per request, up to `max_loras` resident	Yes, same	Yes: `set_adapter` / `disable_adapter()`
Throughput for 1,130 prefills on 8B	~20–30 min on one 80 GB GPU; batching and LoRA switching handled by the scheduler	Similar; `enforce_eager=True` forced, some overhead	Slower; hand-written batching
Olmo-32B	Yes, TP across GPUs	Yes, TP/PP	bf16 fits one 80 GB GPU for forward only
Activations and steering	No	First-class	Hand-written hooks
Already in use here	Yes (behavioural evals)	Same server with the plugin installed	No
Cost of adoption	Lowest: a request builder and a parquet writer	Low: `uv add vllm-lens`, hooks when needed	A second model-loading path, kept to one script

Do this. Write the §4 extractor on Option A: a CPU-side request builder that reads `inputs/*.json`, a single batched vLLM job, a parquet writer per rollout. Check the adapters' rank against `max_lora_rank` and that no adapter touches `lm_head` or the

embeddings (if one does, the base request must use the adapted `lm_head` too, which vLLM multi-LoRA does not do; fall back to Option C for that run). Run the §4.5 validation once on 20 rollouts with Option C and keep that script.

Then switch to vllm-lens, on the same server, for: (1) capturing activations at the KL-spike positions across thousands of rollouts, (2) steering with the directions derived from them, (3) anything on Olmo-32B, (4) running the behavioural evals under steering inside Inspect. If exact KL at scale is ever needed, a persistent hook on the last layer (store base log-probs per request, compute KL on the adapter request, keep the scalars) does it without leaving vLLM.

Pitfalls specific to vLLM here. Request `raw_logprobs` (the default), never `processed_logprobs`, or temperature and penalties leak into the values. Set `detokenize=False` to keep the log-prob post-processing cheap at K = 1,024. Keep `max_num_seqs` modest so 8k-token prefills with 1,024 log-probs each don't exhaust the pod's host memory. Do not read `cumulative_logprob`; it covers the one generated token only.

Not recommended: TransformerLens (a second weight path, weak for 32B), and nnsight for this job (a third stack for no gain; its vLLM backend adds version pinning and no exact-KL path beyond what vllm-lens hooks already give).

6. Analysis

Every analysis is one of four aggregations of the same per-position table: by **position within a rollout** (where), by **region** (planning vs execution), by **token identity across rollouts** (what vocabulary moved), and by **rollout / checkpoint** (when, and for whom). Keep the axis explicit in every plot title.

6.1 Analyses to produce, in order

Three defaults for every analysis below. (1) Run it twice, on rollouts the checkpoint produced (on-policy) and on the rest, and treat a finding as solid only when both agree. (2) Region shares and vocabulary aggregates use `pre_divergence` positions by default, with the all-positions version as a secondary view. (3) Downstream of the divergence, rank positions by `dlp_actual` and rank shift rather than full KL: the log-prob of the written token still separates hack positions from ordinary code after an adapter's whole distribution has gone flat.

1. **Drift series.** Per rollout and pair, total KL and mean KL per completion position (excluding `template`). Plot mean-per-position KL for `plateau`, `surge`, `final` vs `base`, with 95% bootstrap CIs over rollouts, split by `hacked`, with the `step1 vs base`

noise floor as a horizontal line. *Reads:* is there measurable change at `plateau`, i.e. before behaviour moved? Is the jump between `surge` and `final` larger than between `plateau` and `surge`?

2. **Region shares.** For each pair, the fraction of total KL falling in `think`, `answer_prose`, `code`, `test_edit`, `hack_span`. Stacked bars, one bar per pair, split by `hacked`. *Reads:* did RL change planning or execution? Does the share move from `think` toward `code` between checkpoints?
3. **Decision points.** Per rollout and pair: the top-5 positions by `js`, each with ±15 tokens of context, and the cumulative-KL curve against `rel_pos`. Report the `rel_pos` at which cumulative KL crosses 50% ("half-change point"). Compare its distribution for hacked vs non-hacked rollouts. Always include the exit position (last think token, first answer token) in the candidate set: it exists in every rollout, however short the think block, and is where the model commits. *Reads:* a tight cluster of half-change points in `think` means a single early decision carries the change.
4. **Vocabulary movement.** For each pair, sum `p_a·(lp_a − lp_b)` over all positions and rollouts by vocabulary item, using the stored top-20 contribution lists (each listed value is exact or a lower bound, per the `_exact` flag; report what share of each token's total came from bounded values). Rank. Do the same restricted to each region. *Reads:* the ranked list is "what the checkpoint now wants to say"; the negative tail is "what it stopped saying". Hack-technique tokens (skip, conftest, os._exit and the like) at the top of final vs base are the positive control: if they are absent, the pipeline is broken. Keep a hand-built hack lexicon in config and publish a second ranking with the lexicon removed; that residual list, the plateau vs base list (where hacks barely exist yet), and the list restricted to non-hacked W3 rollouts are where anything new will be.
5. **Installed choices.** Positions where `rank_actual_base > 20` and `rank_actual_ckpt == 0`: tokens the checkpoint made its top choice that base would almost never have produced. List with context, grouped by region and by token. Histogram of rank shift. *Reads:* this is the most direct list of RL-written behaviour. Prefer it over contribution mass, which is dominated by frequent code tokens with many small shifts; rank shift isolates positions where a new decision was written in. Run it restricted to think first: that is the quickest test of whether deliberation changed at all.
6. **Asymmetry.** Per region, `kl_ba` − `kl_ab` (base ‖ ckpt minus ckpt ‖ base). *Reads:* when base ‖ ckpt dominates in `think`, the checkpoint has dropped things base used to say there, which is the signature to chase with the no-CoT ablation and logit-lens suppression test.
7. **Entropy.** Mean entropy per region per adapter. *Reads:* sharpening in `test_edit` at `final` = commitment; flattening in `think` = hedging or search.

8. **Position overlap across checkpoints.** Per rollout, Spearman correlation between the `plateau vs base` and `final vs base` KL profiles, and Jaccard overlap of their top-10 positions. *Reads:* high overlap means the same decision points were already moving before the hack surge, which supports the hidden-drift hypothesis; low overlap means `plateau` changed something else.
9. **Deception subset.** For the 63 `disavow_then_hack` rollouts vs the 63 matched controls: KL inside the disavowal sentence (the judge's quoted span) vs inside `hack_span`, per pair. Also `dlp_actual` on the disavowal tokens: does the checkpoint make the disavowal *less* likely than base while making the hack *more* likely? *Reads:* a checkpoint that lowers the disavowal and raises the hack is learning to say less, not to do less.
10. **Consecutive pairs.** Repeat 2, 3 and 4 for `plateau→surge` and `surge→final`. *Reads:* which stage installed which vocabulary and which decision point.

6.2 Visualisations

- **Token-highlighted rollout page** (one HTML per rollout, built for the top-30 rollouts by total KL plus all deception-subset rollouts): tokens coloured by `js` for a chosen pair, hover shows `lp_actual` under each adapter, `rank_actual`, and the top-5 positive and negative contributors. This is the single most useful view; build it first.
- **Rollout × position heatmap** per pair: rows = rollouts sorted by subset then by `hacked`, columns = 100 `rel_pos` bins, cell = mean `js`. Region boundaries drawn as tick marks per row where possible.
- **Drift lines** (analysis 1) and **region stacked bars** (analysis 2).
- **ECDF of per-position `js`** per pair, log x-axis. Shows the heavy tail: most positions near zero, a few spikes. Mark the 95th and 99th percentiles; those thresholds define "spike" everywhere else.
- **Vocabulary volcano plot** per pair: x = mean log-ratio `lp_a − lp_b` over the positions where the token is in either top-20, y = log total contribution magnitude; label the top-30 by magnitude; colour by token class (code keyword, test/pytest-related, grader/evaluation-related, hedging language, other). Companion ranked bar chart of the top-30 positive and top-30 negative contributions.
- **Scatter of per-position KL, `plateau vs base` against `final vs base`**, log–log, one point per position, coloured by region (analysis 8).

Large-vocabulary practice. Never plot the whole vocabulary. Aggregate contributions, rank, show the top-N with labels. Collapse BPE variants before ranking (leading-space vs no-space, case, trailing punctuation) by normalising the decoded string; keep the raw ids in a side table. Separate code tokens from natural-language tokens before comparing,

since code tokens dominate counts. Use log scales for contribution magnitudes. Next to any vocabulary chart, report the share of plotted contribution that is a lower bound rather than exact, so the reader knows how much the ranking leans on bounded values.

6.3 Statistics and controls

- Bootstrap over **rollouts**, never over positions: positions within a rollout are strongly correlated. 1,000 resamples; report medians and 95% intervals.
- Report both total KL and mean per position; long rollouts inflate totals.
- Exclude `template` positions from every aggregate; report their KL separately once, as a sanity check (it should be near zero).
- Noise floor: `step1 vs base`. Any effect below it is not reported as an effect. Also run `base vs base` with a different batch composition to confirm bf16 non-determinism is far below the floor.
- Invariants the pipeline asserts: `kl ⩾ 0`, `kl(a‖a) = 0` within 1e-6, `rest_mass` $\in [0, 1]$ for both adapters, the per-bucket terms over I plus the `rest` term sum to `kl_ab`, `exp(lp)` over each top-K plus `tail_mass` sums to 1 ± 1e-4, and on the §4.5 positions `kl_ab` ⩽ exact KL + 1e-4.
- Judge labels are noisy (59% exact stance agreement on the golden set): treat label-conditional results as trends, and show the deception-subset result on hand-checked rows as well as on the full 63.
- Prompt confound: each step has one prompt, so a `hacked` vs `non-hacked` comparison *across* windows is also a comparison across prompts. Make that comparison only within W2, where both arms exist at the same step.

Primary metrics, fixed in advance: (a) `plateau vs base` mean per-position KL against the noise floor; (b) think-region share of KL at each pair; (c) stability of the top-30 vocabulary across the 1,000 resamples (fraction of resamples in which each token stays in the top-30). Report effect sizes with CIs, no p-values on positions. If (c) is unstable, add rollouts (100 → 500 costs minutes); do not loosen the threshold.

6.4 Reading guide

Pattern	Interpretation	Next experiment
`plateau vs base` above the noise floor, concentrated in `think`	Planning changed before behaviour did (hidden drift)	Extract a direction at those positions; track its projection over all checkpoints

Pattern	Interpretation	Next experiment
Spikes only in `test_edit` / `hack_span`, none in `think`	RL taught execution, not deliberation	Patch the spike positions between checkpoints; check whether the hack is a reflex
`think` installed-choices list empty; everything in `code` / `test_edit`	Deliberation unchanged; nothing hidden in the CoT yet	No-CoT ablation; probe at execution tokens, not the think block
base ‖ ckpt > ckpt ‖ base in `think`	The checkpoint stopped saying something base said	No-CoT ablation; logit-lens suppression check at those positions
Same top positions in `plateau` and `final`	A stable decision point	Activation patching target; probe training positions
Vocabulary top list is grader / evaluation / test words	Grader awareness is in the policy	Build the grader-awareness contrast direction and test it here
Vocabulary list is only hack-technique tokens, even after lexicon removal	Confirmatory only	Shift attention to `plateau vs base` and to non-hacked W3 rollouts
Disavowal tokens lose probability while hack tokens gain	Learning to say less, not do less	Frozen-probe survival test across checkpoints

7. Deliverables and acceptance checks

The agent delivers a small package, a data directory and one report. Each item below has a check the agent runs before calling the item done.

Deliverable	Contents	Done when
`select_rollouts.py`	Builds `manifest.parquet` and `positions.parquet` from the scorer tables and rollout store, per §3	Cell counts printed per window; every exclusion counted; `seed=0` reproduces the same manifest twice

Deliverable	Contents	Done when
`extract_kl.py`	The §4 forward passes and per-position table; `--adapters`, `--pairs`, `--topk`, `--capture-hidden` flags	All §6.3 invariants pass on 5 random rollouts; `tokenization_mismatch` rate reported; runtime logged
`config.yaml`	Steps actually used per checkpoint, tokenizer hash, chat-template settings, dtype, region regexes, versions	Rerunning from the config alone reproduces `kl/final_vs_base.parquet` bit-for-bit on 3 rollouts
`analyze.py`	Produces every figure in §6.2 and the tables behind §6.1, to `figures/` and `tables/`	Each figure's title names its aggregation axis and pair; CIs from rollout-level bootstrap
`rollout_pages/`	Token-highlighted HTML for the top-30 rollouts and the deception subset	Opens offline; hover shows both adapters' `lp_actual`, rank, and top-5 contributors
`report.md`	⩽ 2 pages: drift result with the noise floor, region shares, the top-10 decision points with context, top-30 vocabulary per pair, the deception-subset result, and one paragraph per §6.4 row saying which pattern was seen	Every number traceable to a table file; no claim about mechanism

Order of work. (1) manifest, (2) extractor on 5 rollouts with invariants, (3) full run, (4) rollout pages, (5) the rest of the figures, (6) report. Send the 5-rollout invariant output and one rollout page before the full run. A per-position profile on final vs base that is flat, or uniformly huge from the first token, means a template or tokenisation problem, not a result; fix it before the full run.

Open questions for the agent to resolve and record, not guess:

- Which steps have saved adapters, which adapter modules they cover (attention only, or MLP too; lm_head or embeddings would break multi-LoRA extraction), and their rank.
- Whether the RL system prompt itself mentions tests or grading; if so, note it in the report, since it bounds how "grader-aware" the policy could be without any prompt.
- Where the hack type per rollout is stored, so `hack_span` can be marked.

- Whether completion token ids are stored, or only text (the §4.1 mismatch check depends on it).

Sources

- UKGovernmentBEIS/vllm-lens
- nnsight on PyPI
- vLLM forum: logprob of a specified token
- TorchRL vLLM wrapper notes on prompt logprobs
- Attached: Qwen rollout scoring diagnostics (115-step GRPO run), for the step windows, label definitions and the `disavow_then_hack_63` set